RAMDEOBABA UNIVERSITY, NAGPUR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B.Tech. CSE (Artificial Intelligence and Data Science)

(AIDS)

SEMESTER V

SI LAB
PROJECT REPORT

SMART TRAFFIC ANALYSIS

Submitted by

Aditi Gupta (05)

Enrollment No. : 24RBU/UG/CS/02/000011

Academic Year: 2026–27

SMART TRAFFIC ANALYSIS

SI Lab Project Report

1. PROJECT OBJECTIVE

The objective of this project is to analyze traffic patterns using statistical techniques and machine learning. The project explores traffic-volume data, studies relationships among traffic and environmental variables, tests selected statistical hypotheses, builds regression models for a numerical congestion score, and trains classification models to predict congestion categories. An interactive Gradio interface is included to present dataset summaries and model predictions.

- Load and explore the selected Smart Traffic Analysis dataset.
- Perform detailed exploratory data analysis (EDA) and preprocess the data.
- Demonstrate random sampling and the Central Limit Theorem.
- Analyze correlation and covariance among numerical variables.
- Apply hypothesis testing using a Z-test demonstration, Welch's independent-samples t-test, and one-way ANOVA.
- Build and compare simple and multiple linear regression models.
- Extend linear regression with Ridge and Lasso regularization to compare penalized models against the plain linear baseline.
- Train and evaluate classification models for congestion-level prediction.
- Develop a Gradio user interface for traffic exploration and predictions.

2. INTRODUCTION

Traffic congestion affects travel time, road efficiency, fuel consumption, and the movement of people and goods. Factors such as vehicle volume, road type, time of day, and weather conditions can influence traffic patterns. Analyzing traffic data helps identify these patterns and understand the factors associated with congestion.

This project uses data analysis, statistical techniques, and machine learning to study traffic conditions and make predictions. The dataset contains road characteristics, vehicle counts, timestamps, weather information, congestion scores, and congestion-level labels. Regression and classification models are used to predict congestion, and a Gradio interface provides an interactive way to explore traffic data and view model predictions.

2.1 Problem Statement

The project investigates how traffic volume varies with time, road type, location, and weather, and whether the numerical congestion score and categorical congestion level can be estimated from available traffic and contextual features.

2.2 Scope of the Project

The scope includes data inspection, EDA, cleaning, sampling, CLT demonstration, correlation and covariance analysis, hypothesis testing, linear regression, classification, model evaluation, and a Gradio dashboard. The results depend on the quality and representativeness of the supplied dataset.

3. PROJECT METHODOLOGY

The project follows a staged workflow: load the CSV, inspect its structure, explore patterns visually and statistically, prepare a cleaned dataset, perform statistical analysis, train regression and classification models, compare results, and connect the fitted models to a Gradio interface.

3.1 Dataset Loading and Exploration

The dataset is loaded with pandas using `pd.read_csv(...)`. The notebook checks the dimensions, column names, data types, sample rows, summary statistics, missing values, duplicate records, and categorical distributions.

Dataset dimensions recorded in the notebook: 204,000 rows and 21 columns. No missing values, full duplicate rows, or duplicate `record_id` values were reported in the saved notebook outputs.

The dataset contains identifiers and location fields (`record_id`, `city_zone`, `road_id`, `intersection_id`), road characteristics (`road_type`, `lanes`, `speed_limit`), traffic counts (`vehicle_count`, `motorcycle_count`, `public_transport_count`), time fields (`timestamp`, `hour`, `day_of_week`, `is_weekend`, `is_holiday`), weather fields (`temperature`, `rainfall_mm`, `weather_condition`), and outcomes (`congestion_score`, `fuel_waste_estimate`, `congestion_level`).

Dataset source / URL:
https://www.kaggle.com/datasets/mobeenfatimah/cityflow-smart-urban-mobility-and-traffic-iot

EDA includes numerical distributions, histograms, KDE plots, boxplots, categorical count plots, hourly congestion summaries, weekday/weekend comparisons, weather and road-type comparisons, correlation heatmaps, scatter plots, and grouped summary tables.

Example descriptive finding from the saved notebook: average congestion score rises sharply in the morning hours shown in the grouped summary, from approximately 15.4 –15.5 during hours 0–6 to 60.52 at hour 7, 88.64 at hour 8, and 95.51 at hour 9.

3.2 Data Preprocessing

The preprocessing module converts `timestamp` to datetime, checks and removes duplicate rows if present, handles missing values when required, validates numeric and binary fields, inspects outliers using the interquartile-range (IQR) method, and checks or derives time-related features. Categorical features are encoded where needed. Numerical scaling is applied through model preprocessing pipelines when required.

The notebook keeps the original DataFrame for EDA and creates a cleaned DataFrame for subsequent work. For machine learning, learned transformations should be fitted on the training set and applied to the test set to avoid data leakage.

3.3 Feature Extraction / Selection

Time features include hour, day of week, and weekend/holiday indicators. Regression predicts `congestion_score`; simple linear regression uses `vehicle_count` as its single predictor. Multiple linear regression uses available traffic counts, road characteristics, time indicators, and weather features. Classification predicts `congestion_level` from traffic, road, time, weather, and city-zone features. Identifier columns are not used as predictive features.

The notebook checks for target leakage. It finds that `congestion_level` is directly bucketed from `congestion_score`, so `congestion_score` and `fuel_waste_estimate` are excluded from the classification predictors. The strong relationship between vehicle counts and congestion score is also noted as a possible dataset-engineering limitation.

3.4 Training Multiple Classification Models

Two classification models are implemented and compared. Both use the same target variable and evaluation split for a fair comparison.

Sr. No.	Model Name	Purpose / Notes
1	Logistic Regression	Multiclass baseline classifier; uses a preprocessing pipeline for numerical and categorical inputs.
2	Decision Tree Classifier	Tree-based classifier used to compare predictive performance and interpretability.

3.5 Performance Evaluation and Model Comparison

Classification is evaluated using accuracy, macro-averaged precision, macro-averaged recall, macro F1-score, classification reports, and confusion matrices. The notebook outputs report the following test-set results:

Model	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
Logistic Regression	0.9640	0.9325	0.9238	0.9272
Decision Tree	0.9629	0.9371	0.9141	0.9235

Logistic Regression has slightly higher accuracy, macro recall, and macro F1-score, while the Decision Tree has slightly higher macro precision. Since the target classes are imbalanced, macro-averaged metrics are useful alongside accuracy. The high results should be interpreted in light of the target-generation process and the dataset's provenance.

Regression results recorded in the notebook:

Regression Model	MAE	MSE	RMSE	R^2
Simple Linear Regression	13.949	323.607	17.989	0.7844
Multiple Linear Regression	11.899	230.030	15.167	0.8468
Ridge Regression (alpha=1.0)	11.898	230.30	15.167	0.846
Lasso Regression (alpha=1.0)	12.21	237.03	15.395	0.842

The multiple linear regression model has lower MAE and RMSE and a higher R^2 than the simple model on the saved test split. This is an observed comparison on this dataset, not a guarantee of performance on new locations or future periods.

3.6 Best Model Selection

Based on the saved classification metrics, Logistic Regression is selected as the primary classification model because it has the highest accuracy (0.9640), macro recall (0.9238), and macro F1-score (0.9272) among the two evaluated classifiers. The Decision Tree remains a useful

comparison model. For regression, multiple linear regression performs better than simple linear regression on the saved test split according to MAE, RMSE, and R^2;

Model selection should be revisited after validating the dataset source and target-generation rules. A chronological holdout is particularly important for future traffic prediction because traffic observations are time-dependent.

4. PROJECT IMPLEMENTATION

4.1 Tools and Technologies

Tool / Technology	Version	Purpose
Python	3.13.16	Programming language
Jupyter Notebook / Google Colab	Google Colab	Development and execution environment
Pandas / NumPy	2.2.3	Data handling and numerical operations
Scikit-learn	1.6.1	Preprocessing, regression, classification and metrics
Matplotlib / Seaborn	3.10.0	Data visualization
SciPy	1.16.3	Statistical tests
Gradio	6.30.0	Interactive user interface

4.2 Module-wise Implementation

- Dataset loading and exploration: loads the CSV and inspects the structure, distributions, and quality.
- Data preprocessing: validates fields, handles missing values where needed, and prepares features.
- Statistical analysis: demonstrates sampling, CLT, correlation/covariance, Z-test, Welch's t-test, and one-way ANOVA.
- Regression module: trains simple and multiple linear regression models and evaluates continuous predictions.
- Regularization module: trains Ridge and Lasso regression on the same multiple-regression features and compares their coefficients and error metrics against the plain linear and multiple regression models.
- Classification module: trains Logistic Regression and Decision Tree models and compares classification metrics.
- Model and GUI integration: reuses the fitted regression/classification pipelines and the cleaned dataset in the Gradio interface.

4.3 GUI Implementation

The notebook defines a Gradio dashboard using `gr.Blocks` with tabs for traffic analysis, traffic prediction, and data exploration. The dashboard displays summary statistics and charts, accepts traffic and context features, and reuses the already-fitted multiple linear regression pipeline to estimate `congestion_score` and the Logistic Regression pipeline to estimate `congestion_level`. The interface does not train new models from user inputs.

Figure 1 – Gradio dashboard screenshot:

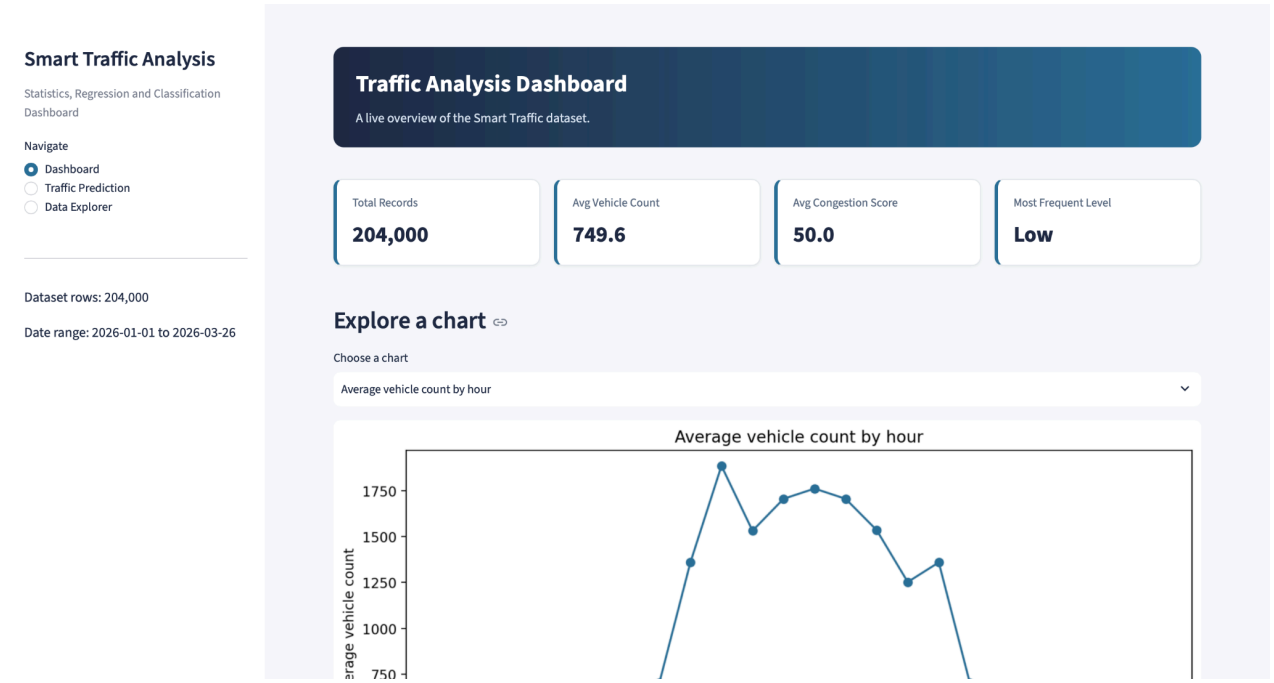
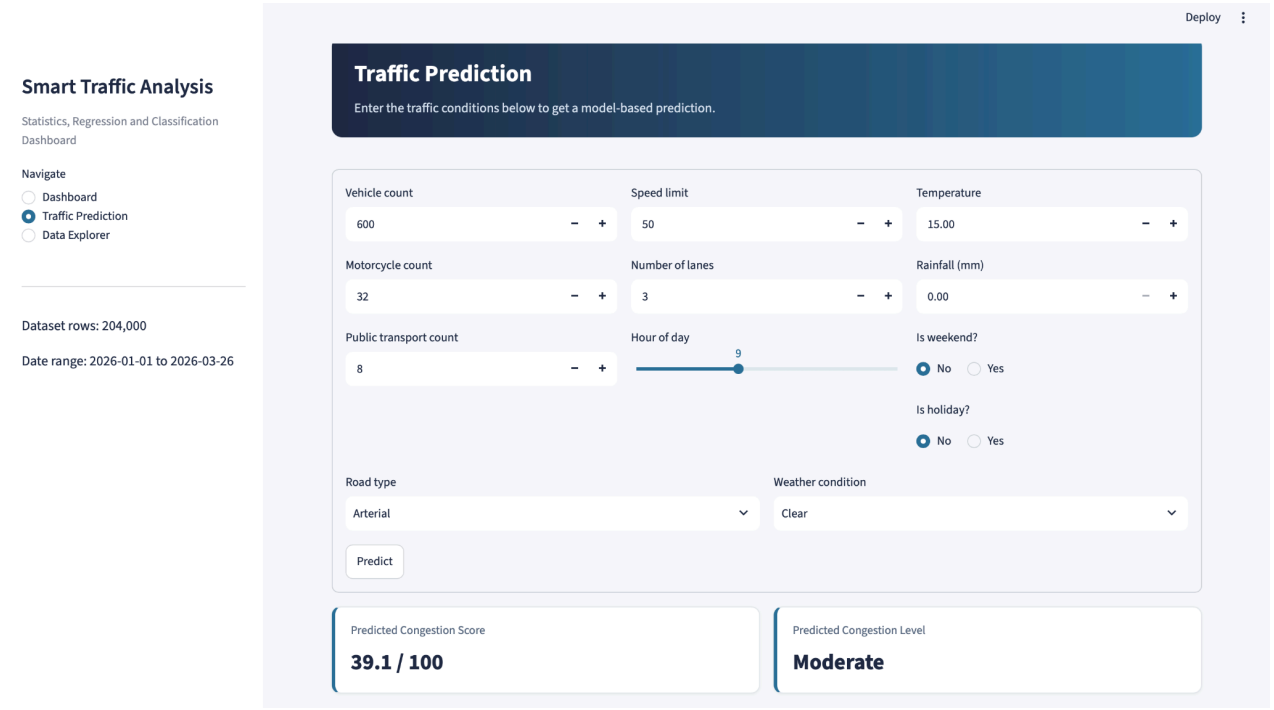


Figure 2 – Example prediction screenshot:



5. COMPLETE CODE

The complete implementation is provided in the Jupyter Notebook named `Smart_Traffic_Analysis.ipynb`. It contains the dataset-loading cell, EDA, preprocessing, statistical tests, regression models, classification models, evaluation tables, visualizations, and Gradio UI code.

Notebook / Colab URL:
https://colab.research.google.com/drive/1NBNkq3EqV6YPYWpPHrwDX-lOdFFFW90i?usp=sharing

6. RESULTS AND DISCUSSION

The dataset contains 204,000 observations and 21 columns, with no missing values, fully duplicated rows, or duplicate record identifiers reported in the saved notebook outputs. Exploratory Data Analysis (EDA) revealed variations in congestion across different hours and weather conditions. The average congestion score ranged from 49.63 during Clear weather to 56.88 during Heavy Rain.

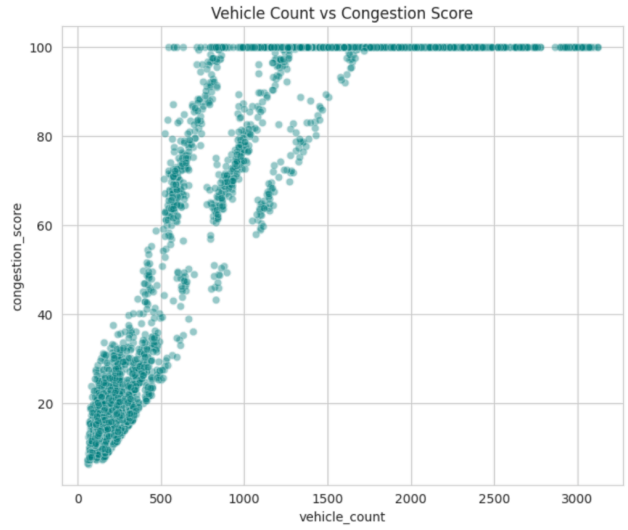
Welch's t-test comparing weekday and weekend vehicle counts returned a t-statistic of 101.5277 and a p-value displayed as 0.000000 (rounded). At a significance level of 0.05, the null hypothesis was rejected. One-way ANOVA across weather conditions returned an F-statistic of 1.6290 and a p-value of 0.180275; therefore, the null hypothesis was not rejected. The one-sample Z-test reported a Z-statistic of 0.9700 and a p-value of 0.3320. Since the sample and reference statistics came from the same dataset, this result is treated as an illustrative exercise rather than evidence about an external population.

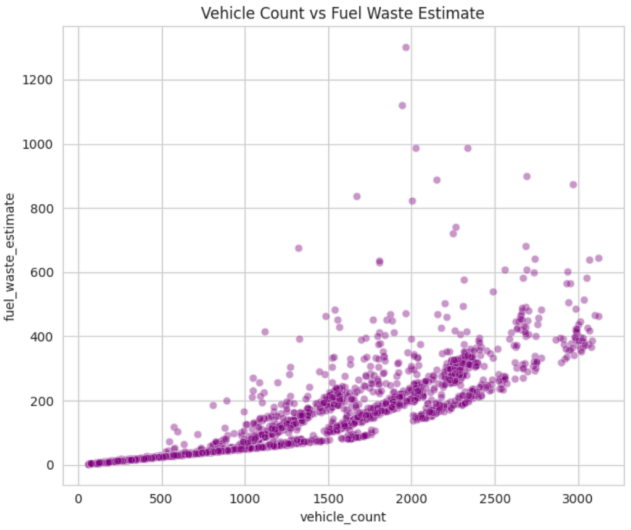
For regression, Simple Linear Regression achieved an MAE of 13.949, RMSE of 17.989, and R^2 of 0.7844. Multiple Linear Regression performed better, with an MAE of 11.899, RMSE of 15.167, and R^2 of 0.8468. Ridge and Lasso Regression were also included to compare regularized models with standard linear regression. Their performance should be evaluated using the final metrics obtained from the notebook.

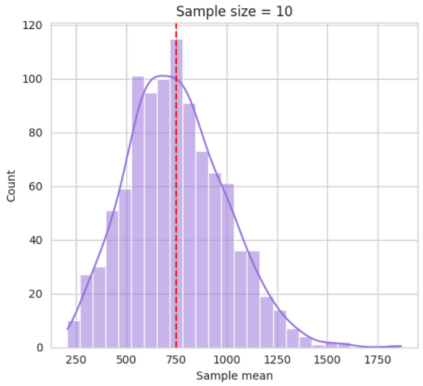
For classification, Logistic Regression achieved 96.40% accuracy and a macro F1-score of 92.72%, while the Decision Tree achieved 96.29% accuracy and a macro F1-score of 92.35%. Logistic Regression performed slightly better overall and was selected as the primary classification model.

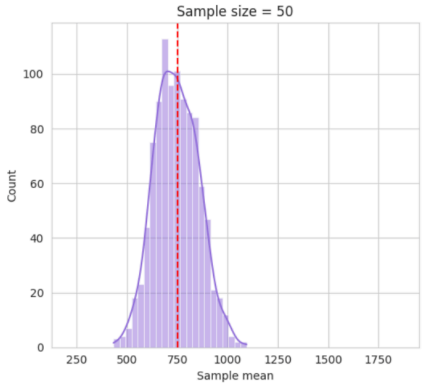
The correlation and covariance analyses helped identify relationships between numerical traffic variables. The comparison of regression models showed that using multiple input features improved prediction performance compared with Simple Linear Regression. The classification models also demonstrated strong performance in predicting congestion categories. The Gradio interface made the project more interactive by allowing users to view traffic information and obtain predictions. Overall, the project demonstrates how statistical analysis and machine learning can support traffic pattern analysis and congestion prediction. These findings are specific to the selected dataset and test split.

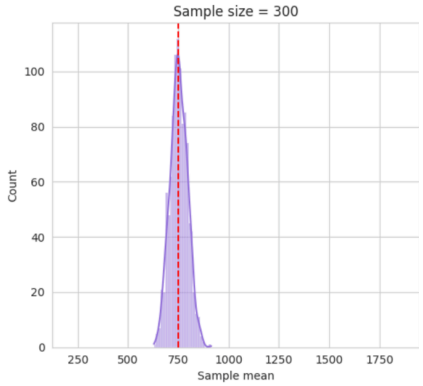
6.1 Required Output Screenshots

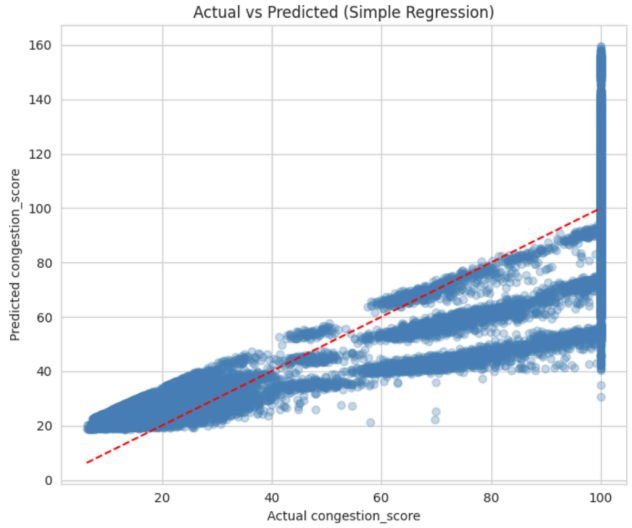


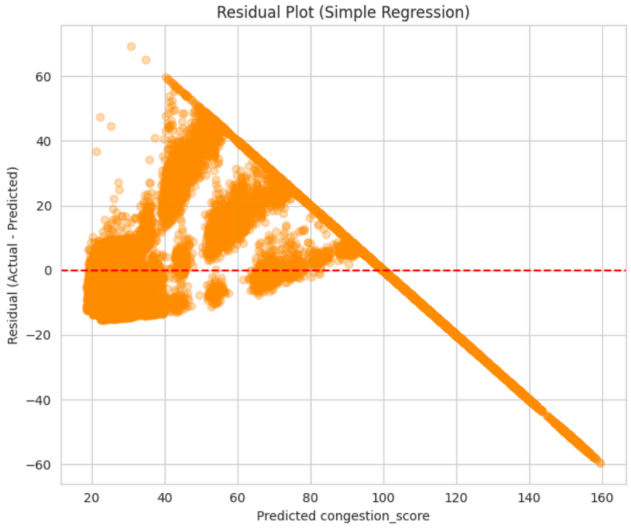












```
comparison_table = pd.DataFrame({
    "Model": [
        "Simple Linear Regression (vehicle_count only)",
        "Multiple Linear Regression (all predictors)",
        "Ridge Regression (alpha=1.0)",
        "Lasso Regression (alpha=1.0)",
    ],
    "MAE": [mae_s, mae_m, mae_ridge, mae_lasso],
    "MSE": [mse_s, mse_m, mse_ridge, mse_lasso],
    "RMSE": [rmse_s, rmse_m, rmse_ridge, rmse_lasso],
    "R2": [r2_s, r2_m, r2_ridge, r2_lasso],
})
comparison_table
```

	Model	MAE	MSE	RMSE	R2
0	Simple Linear Regression (vehicle_count only)	13.949174	323.607285	17.989088	0.784414
1	Multiple Linear Regression (all predictors)	11.898617	230.030257	15.166748	0.846755
2	Ridge Regression (alpha=1.0)	11.898667	230.030434	15.166754	0.846755
3	Lasso Regression (alpha=1.0)	12.212745	237.030059	15.395781	0.842091

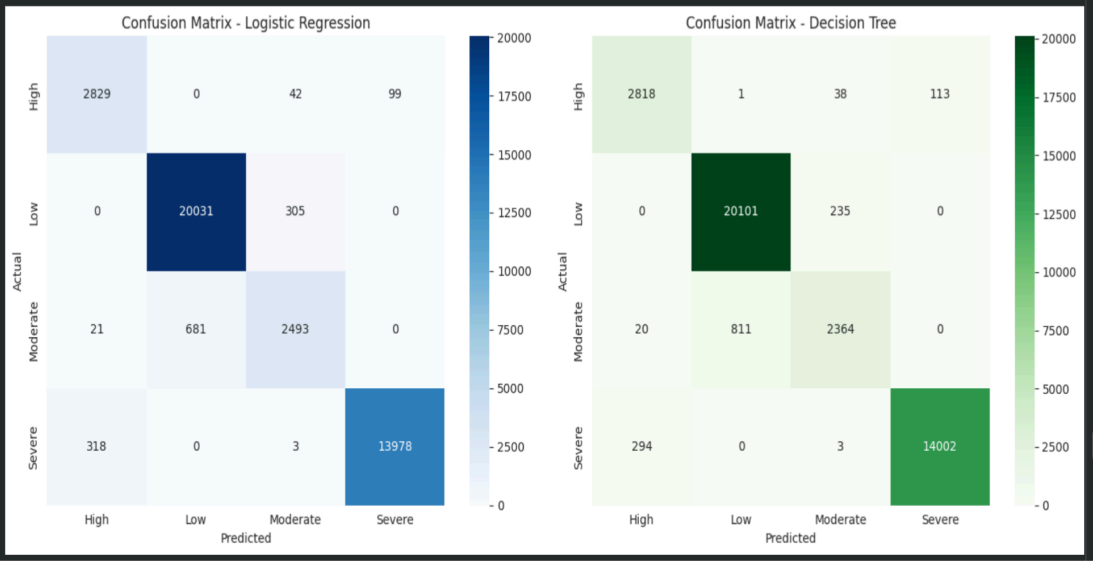
```
model_comparison_clf = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree"],
    "Accuracy": [logreg_scores[0], tree_scores[0]],
    "Precision (macro)": [logreg_scores[1], tree_scores[1]],
    "Recall (macro)": [logreg_scores[2], tree_scores[2]],
    "F1-score (macro)": [logreg_scores[3], tree_scores[3]],
})
model_comparison_clf
```

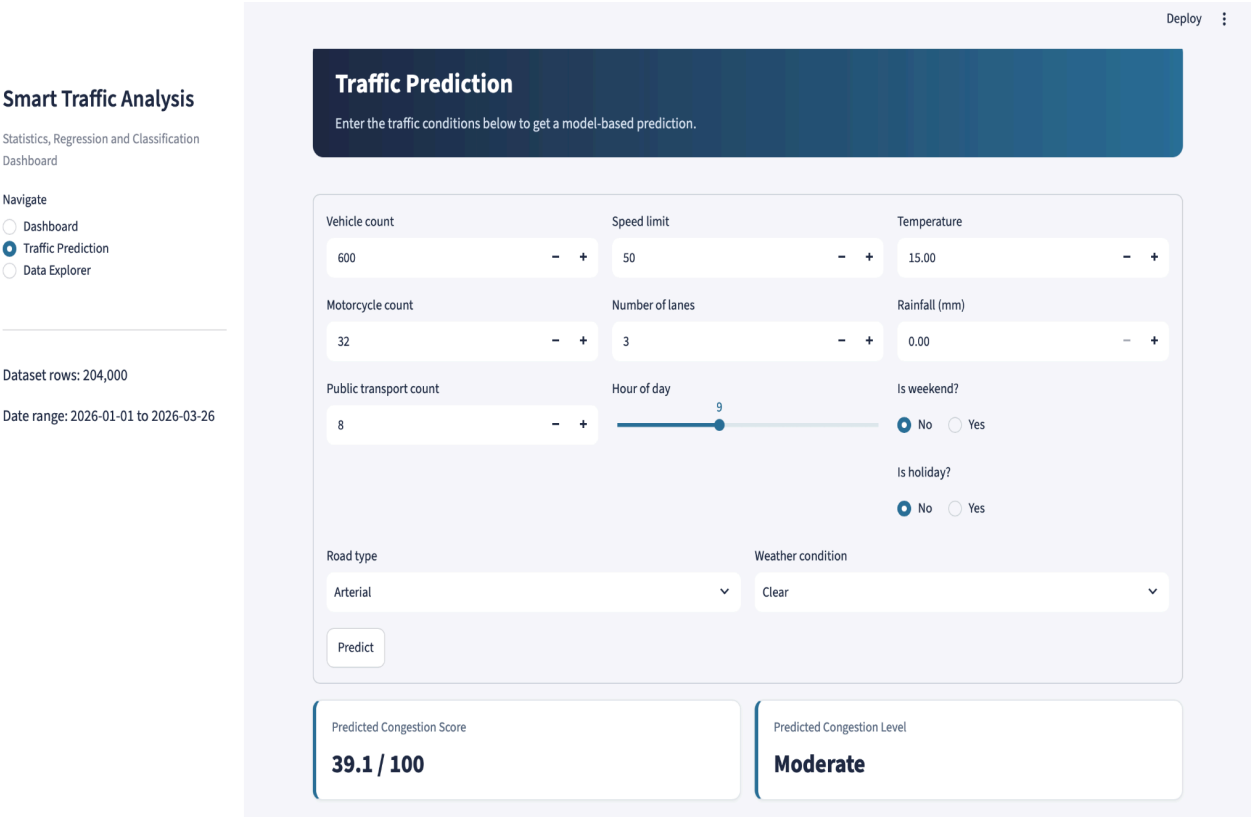
	Model	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
0	Logistic Regression	0.963995	0.932493	0.92384	0.927187
1	Decision Tree	0.962868	0.937091	0.91410	0.923526

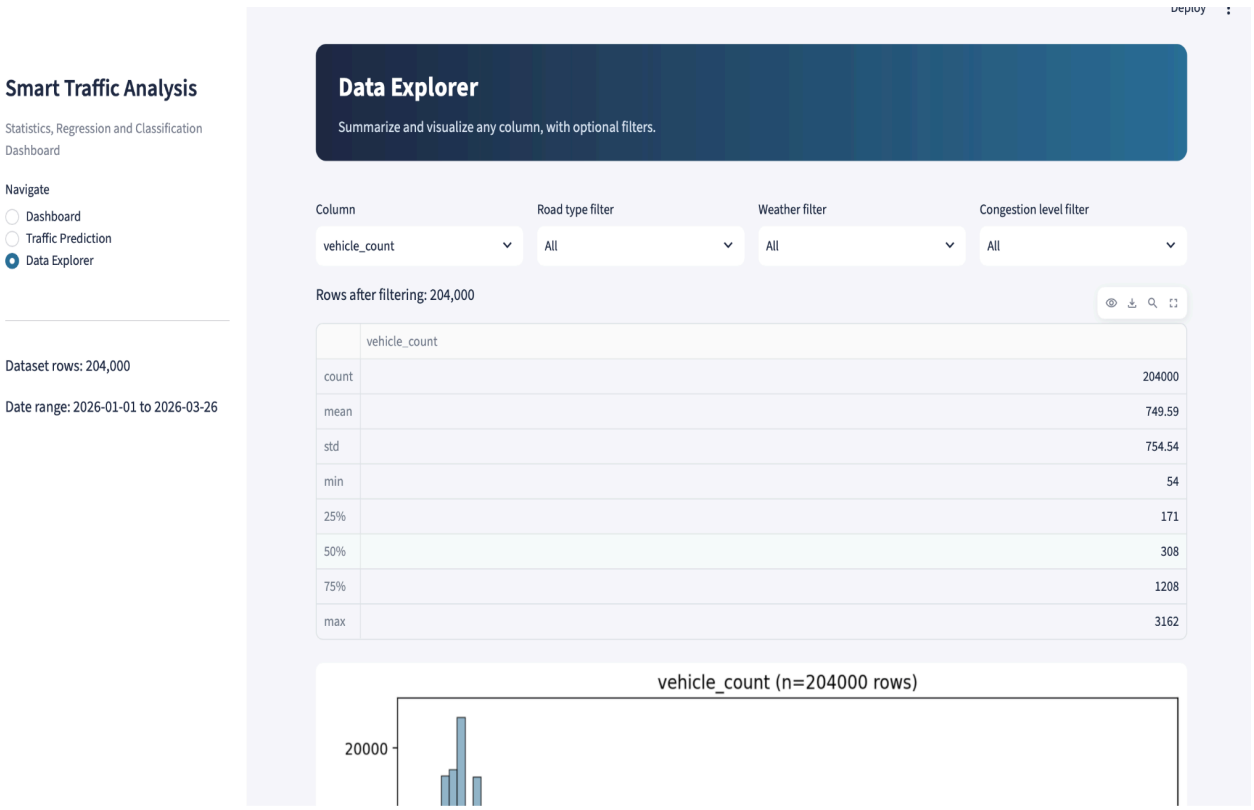
```
axes[0].set_ylabel("Actual")

cm_tree = confusion_matrix(y_test_c, y_pred_tree, labels=class_labels)
sns.heatmap(cm_tree, annot=True, fmt="d", cmap="Greens", xticklabels=class_labels, yticklabels=class_labels, ax=
axes[1].set_title("Confusion Matrix - Decision Tree")
axes[1].set_xlabel("Predicted")
axes[1].set_ylabel("Actual")

plt.tight_layout()
plt.show()
```




Deploy
Smart Traffic Analysis
Statistics, Regression and Classification Dashboard
Navigate
Dashboard
Traffic Prediction
Data Explorer
Dataset rows: 204,000
Date range: 2026-01-01 to 2026-03-26
Traffic Prediction
Enter the traffic conditions below to get a model-based prediction.
Vehicle count
600
Speed limit
50
Temperature
15.00
Motorcycle count
32
Number of lanes
3
Rainfall (mm)
0.00
Public transport count
8
Hour of day
9
Is weekend?
No
Yes
Is holiday?
No
Yes
Road type
Arterial
Weather condition
Clear
Predict
Predicted Congestion Score
39.1 / 100
Predicted Congestion Level
Moderate


Smart Traffic Analysis
Statistics, Regression and Classification Dashboard
Navigate
Dashboard
Traffic Prediction
Data Explorer
Dataset rows: 204,000
Date range: 2026-01-01 to 2026-03-26
Data Explorer
Summarize and visualize any column, with optional filters.
Column
vehicle_count
Road type filter
All
Weather filter
All
Congestion level filter
All
Rows after filtering: 204,000
vehicle_count
count 204000
mean 749.59
std 754.54
min 54
25% 171
50% 308
75% 1208
max 3162
vehicle_count (n=204000 rows)
20000

6.2 Result Analysis

The multiple regression model performs better than the simple regression model on the saved test split. Logistic Regression has slightly better overall accuracy and macro F1-score than the Decision Tree, although the Decision Tree has slightly higher macro precision. The classification target is imbalanced: Low and Severe are much more frequent than Moderate and High. Macro metrics therefore provide useful additional context.

The congestion category is a direct bucketing of congestion score. Excluding the score and fuel-waste estimate from the classification features reduces direct leakage, but vehicle counts may still be related to how the categories were generated. In addition, repeated observations over time can violate the independence assumptions of classical tests. These limitations should be stated during the viva and addressed in future work.

7. CONCLUSION

This project demonstrates the application of exploratory data analysis, data preprocessing, statistical techniques, regression, and classification to traffic data. Simple Linear Regression, Multiple Linear Regression, Ridge Regression, and Lasso Regression were evaluated for predicting the congestion score. Multiple Linear Regression and Ridge Regression achieved similar results on the saved test split, while Lasso Regression showed slightly higher prediction errors.

For congestion-level classification, Logistic Regression achieved higher accuracy, macro recall, and macro F1-score than the Decision Tree Classifier. The project also includes a Gradio interface for exploring traffic data and generating predictions.

The results are specific to the selected dataset and evaluation split. Future improvements could include validating the dataset source, evaluating models on later time periods, and testing their performance on independent traffic observations.

8. GITHUB LINK

GitHub Repository URL:
https://github.com/aditigupta217/SI-SMART-TRAFFIC-ANALYSIS

9. REFERENCES

[1] Dataset source:

https://www.kaggle.com/datasets/mobeenfatimah/cityflow-smart-urban-mobility-and-traffic-iot

[2] Pandas documentation. https://pandas.pydata.org/docs/

[3] SciPy documentation. https://docs.scipy.org/doc/scipy/

[4] Scikit-learn documentation. https://scikit-learn.org/stable/

[5] Matplotlib documentation. https://matplotlib.org/stable/

[6] Seaborn documentation. https://seaborn.pydata.org/

[7] Gradio documentation. https://www.gradio.app/docs/